

Building a GET API Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-building-get-api-lab>  <https://github.com/learn-co-curriculum/python-p4-building-get-api-lab/issues/new>

Learning Goals

- Build an API to handle GET requests.

Key Vocab

- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.

Instructions

This is a **test-driven lab**.

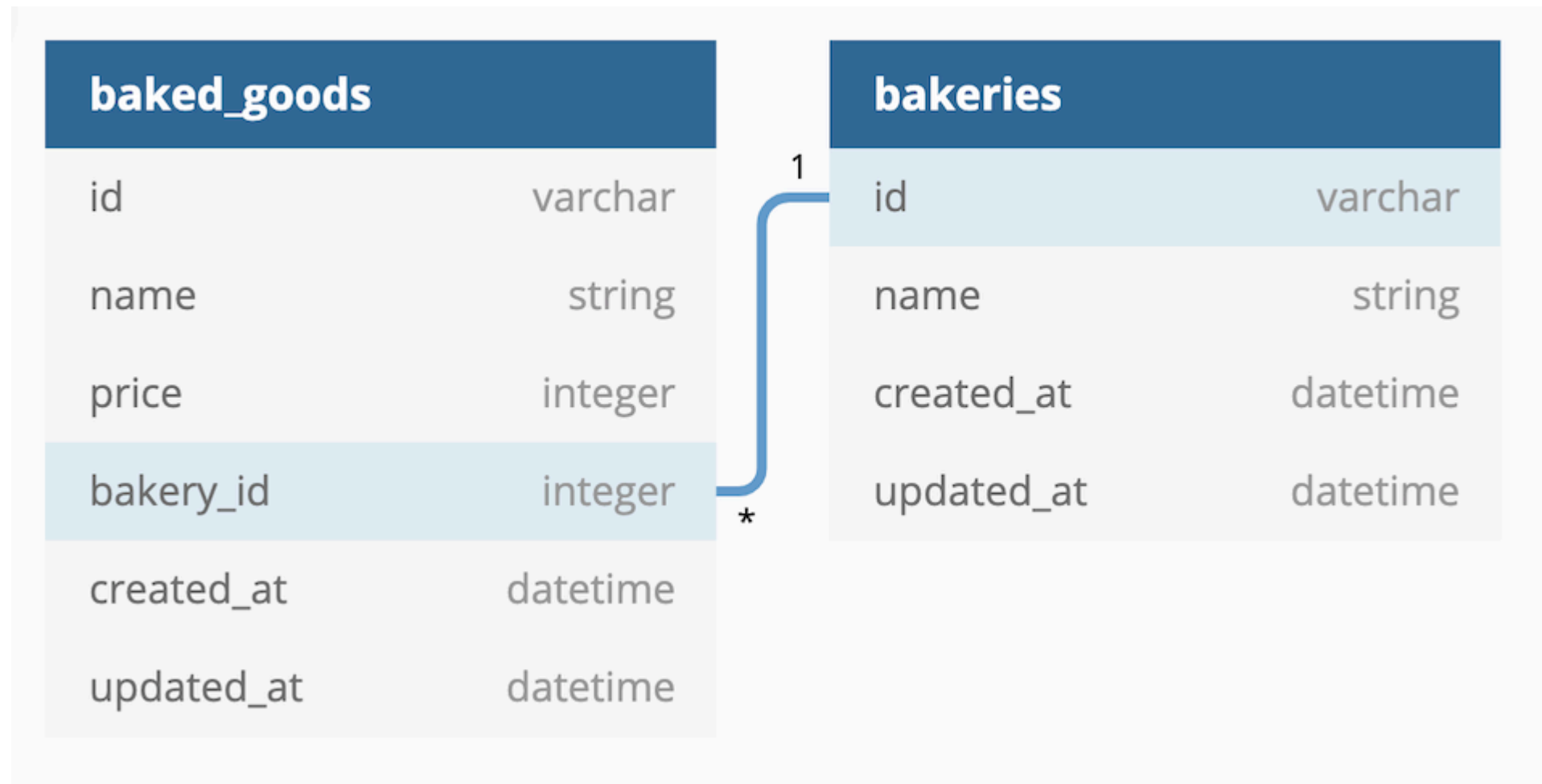
Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

In this application, we'll be working on a JSON API to get a list of bakeries and their baked goods. We have two models, bakeries and baked goods, that have a one-to-many relationship. Here's what the ERD for these tables looks like:



The commands `flask db init` and `flask db migrate` have already been run. Run the following command to initialize the database from the existing migration script:

```
$ flask db upgrade head
```

Run the following command to seed the table with sample data:

```
$ python seed.py
```

You can run the app and explore your API in the browser by using Flask command:

```
$ flask run
```

Edit `app.py` to handle the following `GET` requests:

- `GET /bakeries` : returns a list of JSON objects for all bakeries in the database.
- `GET /bakeries/<int:id>` : returns a single bakery as JSON with its baked goods nested in a list. Use the `id` from the URL to look up the correct bakery.
- `GET /baked_goods/by_price` : returns a list of baked goods as JSON, sorted by price in **descending** order. (**HINT**: how can you use SQLAlchemy to sort the baked goods in descending order?)
- `GET /baked_goods/most_expensive` : returns the single most expensive baked good as JSON. (**HINT**: how can you use SQLAlchemy to sort the baked goods in **descending** order *and* limit the number of results?)

Once all of your tests are passing, commit and push your work using `git` to submit.

Examples

All Bakeries `http://127.0.0.1:5555/bakeries`

```
[  
  {  
    "baked_goods": [  
      {
```

```
    "bakery_id": 1,
    "created_at": "2023-11-03 16:11:18",
    "id": 1,
    "name": "Chocolate dipped donut",
    "price": 2.75,
    "updated_at": null
  },
  {
    "bakery_id": 1,
    "created_at": "2023-11-03 16:11:18",
    "id": 2,
    "name": "Apple-spice filled donut",
    "price": 3.5,
    "updated_at": null
  }
],
"created_at": "2023-11-03 16:11:18",
"id": 1,
"name": "Delightful donuts",
"updated_at": null
},
{
  "baked_goods": [
    {
      "bakery_id": 2,
      "created_at": "2023-11-03 16:11:18",
      "id": 3,
      "name": "Glazed honey cruller",
      "price": 3.25,
      "updated_at": null
    },
    {
      "bakery_id": 2,
```

```
    "created_at": "2023-11-03 16:11:18",
    "id": 4,
    "name": "Chocolate cruller",
    "price": 100,
    "updated_at": "2023-11-03 16:11:18"
  }
],
"created_at": "2023-11-03 16:11:18",
"id": 2,
"name": "Incredible crullers",
"updated_at": null
}
```

Bakery by ID <http://127.0.0.1:5555/bakeries/1>

```
{
  "baked_goods": [
    {
      "bakery_id": 1,
      "created_at": "2023-11-03 16:11:18",
      "id": 1,
      "name": "Chocolate dipped donut",
      "price": 2.75,
      "updated_at": null
    },
    {
      "bakery_id": 1,
      "created_at": "2023-11-03 16:11:18",
      "id": 2,
      "name": "Apple-spice filled donut",
      "price": 3.5,
      "updated_at": null
    }
  ]
}
```

```
}
],
"created_at": "2023-11-03 16:11:18",
"id": 1,
"name": "Delightful donuts",
"updated_at": null
}
```

Baked Goods By Price http://127.0.0.1:5555/baked_goods/by_price

```
[
{
  "bakery": {
    "created_at": "2023-11-03 16:31:32",
    "id": 1,
    "name": "Delightful donuts",
    "updated_at": null
  },
  "bakery_id": 1,
  "created_at": "2023-11-03 16:31:32",
  "id": 2,
  "name": "Apple-spice filled donut",
  "price": 3.5,
  "updated_at": null
},
{
  "bakery": {
    "created_at": "2023-11-03 16:31:32",
    "id": 2,
    "name": "Incredible crullers",
    "updated_at": null
  },
  "bakery_id": 2,
```

```
"created_at": "2023-11-03 16:31:32",
"id": 4,
"name": "Chocolate cruller",
"price": 3.4,
"updated_at": null
},
{
  "bakery": {
    "created_at": "2023-11-03 16:31:32",
    "id": 2,
    "name": "Incredible crullers",
    "updated_at": null
  },
  "bakery_id": 2,
  "created_at": "2023-11-03 16:31:32",
  "id": 3,
  "name": "Glazed honey cruller",
  "price": 3.25,
  "updated_at": null
},
{
  "bakery": {
    "created_at": "2023-11-03 16:31:32",
    "id": 1,
    "name": "Delightful donuts",
    "updated_at": null
  },
  "bakery_id": 1,
  "created_at": "2023-11-03 16:31:32",
  "id": 1,
  "name": "Chocolate dipped donut",
  "price": 2.75,
  "updated_at": null
}
```

```
}  
]
```

Most Expensive Baked Good http://127.0.0.1:5555/baked_goods/most_expensive

```
{  
  "bakery": {  
    "created_at": "2023-11-03 16:16:21",  
    "id": 1,  
    "name": "Delightful donuts",  
    "updated_at": null  
  },  
  "bakery_id": 1,  
  "created_at": "2023-11-03 16:16:21",  
  "id": 2,  
  "name": "Apple-spice filled donut",  
  "price": 3.5,  
  "updated_at": null  
}
```

Resources

- [Flask - Pallets](https://flask.palletsprojects.com/en/2.2.x/)  (https://flask.palletsprojects.com/en/2.2.x/)
- [GET - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)  (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)
- [flask.json.jsonify Example Code - Full Stack Python](https://www.fullstackpython.com/flask-json-jsonify-examples.html)  (https://www.fullstackpython.com/flask-json-jsonify-examples.html)
- [SQLAlchemy-serializer - PyPI](https://pypi.org/project/SQLAlchemy-serializer/)  (https://pypi.org/project/SQLAlchemy-serializer/)

This tool needs to be loaded in a new browser window

Load Building a GET API Lab (CodeGrade) in a new window

